

Twitty · RTL Terminal

A bidirectional terminal emulator for mixed Arabic and English text. Arabic shapes contextually and flows right-to-left, inline with English, inside real shells and TUI applications.

مرحباً بك في Twitty — طرفية تدعم العربية والإنجليزية معاً العربية تتشكّل وتُعرض من اليمين لليسار

What It Does

Twitty runs a real shell (PowerShell, Bash, Zsh) inside a pseudo-terminal and renders it with xterm.js. Mixed Arabic/English text is **contextually shaped** (letters take their correct initial, medial, final, or isolated forms and join) and **ordered right-to-left** per Arabic run — without ever rewriting the byte stream, so cursor positioning and TUI applications stay correct.

Contextual Arabic Shaping

Letters join with correct presentation forms (U+FE70–FEFF). Bundled Noto Naskh Arabic font ensures rendering on every OS.

Per-Run Bidirectional Ordering

Arabic flows right-to-left; English, digits, and symbols stay left-to-right, mixed on the same line.

Render-Layer Architecture

Shaping happens as the browser draws cells, not in the byte stream. Buffer stays logical — cursor never desyncs.

TUI-Safe

Works inside alternate-screen apps (editors, pagers). Box-drawing and block glyphs tile with seamless row alignment.

Cross-Platform

Windows (ConPTY), Linux (Unix PTY). Single CI workflow produces NSIS installer + .deb / .rpm / Appliance.

Full ANSI Support

24-bit color, alternate screen, scroll regions, mouse reporting, 10 000-line scrollbar.

Arabic Rendering in Action

Twitty · RTL-first terminal · browser demo (no PTY attached)

Arabic shapes contextually and flows right-to-left, inline with English.
English stays LTR · العربية تتشكل وتُعرض من اليمين لليسار /
طرفية تدعم العربية والإنجليزية معا - Twitty مرحبا بك في

> git status # الفرع: main · نظيف
> echo "السلام عليكم, world"
السلام عليكم, world

Box-drawing + block glyphs render crisply, so TUIs and borders stay intact:

Capability	Status
Contextual Arabic shaping	✓ live
Per-run BiDi ordering	✓ live
GPU box-drawing glyphs	✓ live

Run `npm run tauri:dev` to launch the real terminal.

• Demo (no PTY)

AR ⇌ EN

F11

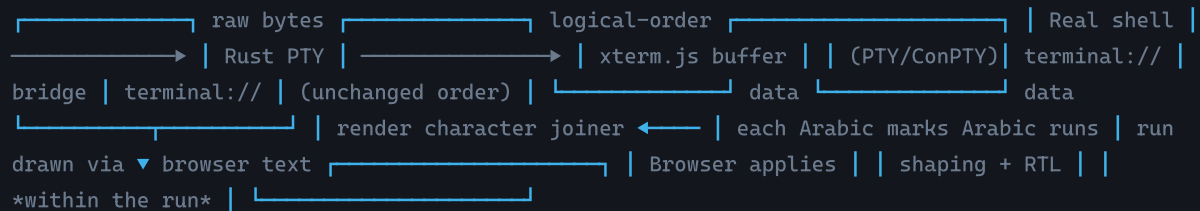
fullscreen

Ctrl+C

interrupt

Architecture

The Rust backend spawns a real shell inside a PTY and streams raw output to the frontend via Tauri events. xterm.js handles ANSI parsing, cursor movement, colors, and scrollbar. Keystrokes flow back via `write_terminal`; window resizes via `resize_terminal`.



How the RTL Rendering Works

The hard part of an Arabic terminal isn't shaping — it's shaping **without corrupting the terminal**. Stream-rewriting approaches change cell counts and columns, which desyncs the cursor and produces duplicated text in TUI apps. Twitty avoids this entirely:

1. **The byte stream is never modified.** Raw PTY output is written into xterm.js exactly as received, so the buffer stays in logical order.
2. **A character joiner marks Arabic runs.** A run begins and ends on an Arabic letter and may include spaces between words, so multi-word phrases order RTL together. The regex matches `[\u0600-\u06FF...] (? : [\u0600-\u06FF... ]* [\u0600-\u06FF...])?`.
3. **The DOM renderer draws each run through the browser's text engine**, which applies native contextual shaping and right-to-left ordering inside the run only.

Because reordering happens at draw time and not in the buffer, cursor math, scroll regions, and alternate-screen redraws all stay correct.

العربية تتشكّل وتُعرض من اليمين لليسار داخل سطر الطرفية · الإنجليزية تبقى من اليسار لليمين · المواقع لا تتأثر

Key Files

File

Responsibility

`XtermTerminal.tsx`

xterm.js setup, Arabic character joiner, key handling

File	Responsibility
<code>StatusBar.tsx</code>	Live connection / shell / AR↔EN status bar
<code>pty.rs</code>	PTY session lifecycle and raw I/O
<code>lib.rs</code>	Tauri commands: start, write, interrupt, resize, stop
<code>build.yml</code>	GitHub Actions: Windows + Linux builds and release

Tech Stack

Layer	Technology
Shell	Tauri v2 (Rust core, system WebView)
Backend	Rust + <code>portable-pty</code> (ConPTY + Unix PTY)
Frontend	React 19 + Vite 6 + xterm.js 6 + addon-fit
Arabic Font	Noto Naskh Arabic (SIL OFL), bundled woff2

Platform Support

Platform	WebView	Build Output
Windows	WebView2 (Chromium)	NSIS installer
Linux	webkit2gtk (WebKit)	.deb / .rpm / Applmage
macOS	WebKit	Buildable from source

Quick Start

Browser demo (no shell)

```
npm install
npm run dev
```

Open the printed localhost URL. Shows a demo banner since there is no PTY in the browser.

Desktop app (real shell)

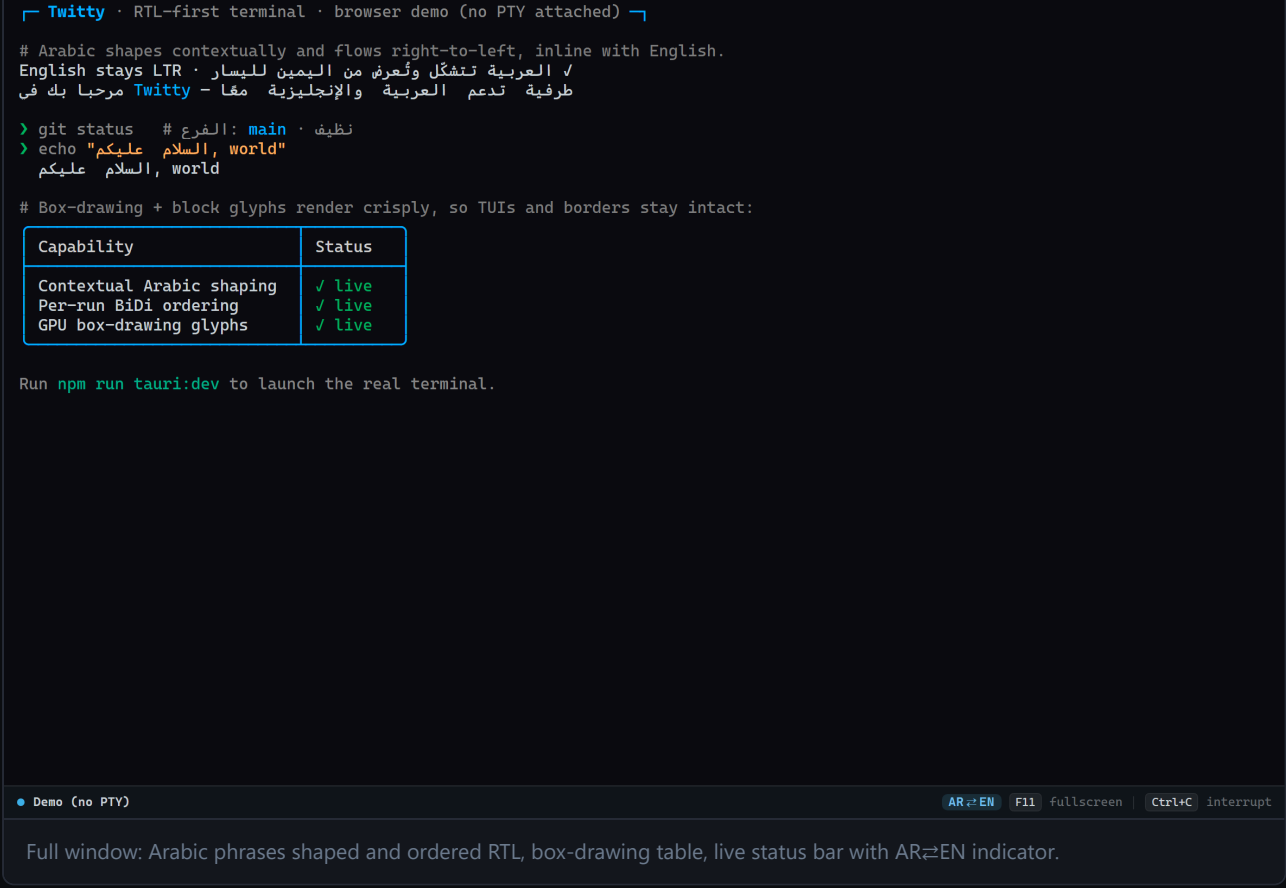
```
npm install
npm run tauri:dev
```

Requires Rust. Windows launches PowerShell; Unix uses \$SHELL, falling back to /bin/bash.

Production build

```
npm run build
npm run tauri:build
```

Screenshots



Design Decisions

Why the DOM renderer, not WebGL? xterm.js offers a WebGL renderer that rasterizes text into GPU glyph atlases. However, its atlas clips Arabic glyph overhang strokes (like the descender arm of ك) and inserts wide gaps between words in joined runs. The DOM renderer draws Arabic through the browser's native text layout engine, which shapes and orders it correctly. Box-drawing and block glyphs still tile seamlessly with `lineHeight: 1.0` and a proper monospace font stack.

Why render-layer, not stream-layer? Earlier approaches rewrote PTY output before handing it to the renderer. This changed cell counts and desynced cursor positioning, producing duplicated and misplaced text in TUI apps. By moving shaping to the render layer (via `registerCharacterJoiner`), the buffer stays in logical order and all terminal semantics remain correct.